# Web应用开发C

成都信息工程大學
Chengdu University of Information Technology

学校：成都信息工程大学

学院：软件工程学院

专业：数据科学与大数据技术

授课教师：杨与广

邮箱：yyg@cuit.edu.cn

群名称:数科221-Web应用开发C
群　号:870648816

# 第四章 JavaScript 回顾



- JavaScript 概述　　组成、作用、特点
- JavaScript 基础　　基本用法、调试、基本语法
- BOM　　概念、window、document、location、history、screen、navigator
- DOM　　概念、节点关系、节点访问、节点操作
- jQuery　　作用、安装、基本语法、选择器、事件
- Bootstrap　　作用、安装、基本用法、各种样式设置

# 第五章 Flask-安装及基本结构



- 安装
- 应用的基本结构

# 第五章 Flask-安装及基本结构



- 安装
- 应用的基本结构

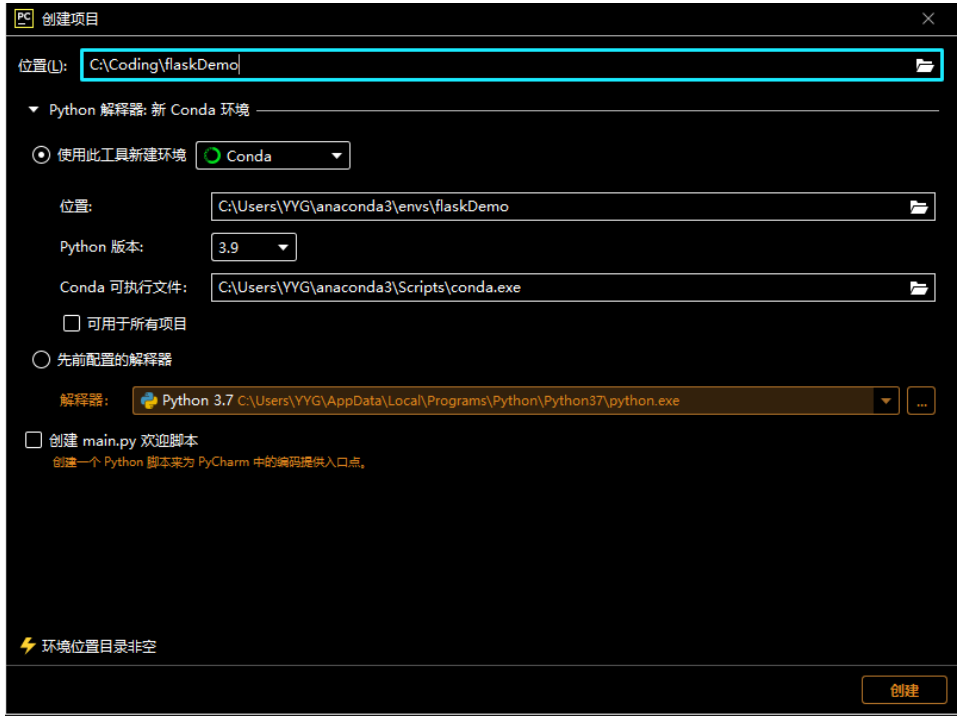
# 安装

成都信息工程大學 Chengdu University of Information Technology

## 环境配置

https://blog.csdn.net/m0_46374969/article/details/131292897
https://blog.csdn.net/m0_61607990/article/details/129531686

- 创建应用目录　　用于存放项目原码及文件
- 配置虚拟环境　　用于配置项目python解释器的私有环境、推荐Anaconda
- 安装依赖包　　项目所依赖的python包，pip install 包名




```
PS C:\Coding\flaskDemo> pip install flask
```

Tabnine

```
PS C:\Coding\flaskDemo> pip freeze
blinker==1.6.2
click==8.1.7
colorama==0.4.6
Flask==3.0.0
importlib-metadata==6.8.0
itsdangerous==2.1.2
Jinja2==3.1.2
MarkupSafe==2.1.3
Werkzeug==3.0.0
zipp==3.17.0
```

https://zhuanlan.zhihu.com/p/338280181?utm_id=0

# 第五章 Flask-安装及基本结构




成都信息工程大學
Chengdu University of Information Technology

- 安装
- 应用的基本结构

# 应用的基本结构

成都信息工程大學 Chengdu University of Information Technology

https://flask.net.cn/ https://www.bilibili.com/video/BV17z4y1X7UZ

- Hello World
  - 导入Flask
  - 新建应用实例
  - 创建路由
  - 运行程序

```python
# 1、导入所需的python包
from flask import Flask

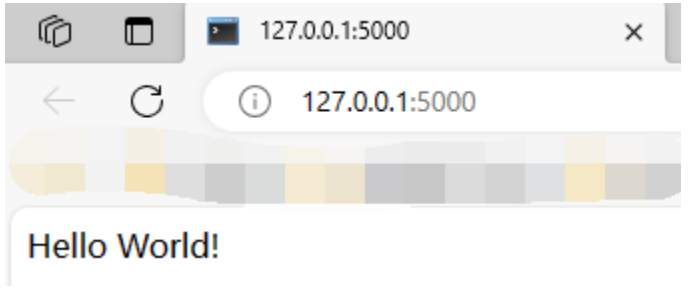
# 2、创建一个flask应用实例
app = Flask(__name__)


# 3、创建路由
@app.route('/')
def hello():
    return 'Hello World!'


# 4、程序运行入口
if __name__ == '__main__':
    app.run(debug=True)

```

```
运行: app ×
C:\Users\YYG\anaconda3\envs\flaskDemo\python.exe C:/Coding/flaskDemo/app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 142-598-493
```



# 应用的基本结构



https://www.bilibili.com/video/BV17z4y1X7UZ

## 初始化

- 创建一个Flask应用实例，Web服务器使用WSGI(Web server gateway interface)协议，将接收到的来自客户端的所有请求转交给所创建的这个对象处理。
- 实现：

```
from flask import Flask
app = Flask(__name__)
```

```
import_name=import_name,
static_url_path=static_url_path,
static_folder=static_folder,
static_host=static_host,
host_matching=host_matching,
subdomain_matching=subdomain_matching,
template_folder=template_folder,
instance_path=instance_path,
instance_relative_config=instance_relative_config,
root_path=root_path,
```

```
"DEBUG": None,
"TESTING": False,
"PROPAGATE_EXCEPTIONS": None,
"SECRET_KEY": None,
"PERMANENT_SESSION_LIFETIME": timedelta(days=31),
"USE_X_SENDFILE": False,
"SERVER_NAME": None,
"APPLICATION_ROOT": "/",
"SESSION_COOKIE_NAME": "session",
"SESSION_COOKIE_DOMAIN": None,
"SESSION_COOKIE_PATH": None,
"SESSION_COOKIE_HTTPONLY": True,
"SESSION_COOKIE_SECURE": False,
"SESSION_COOKIE_SAMESITE": None,
"SESSION_REFRESH_EACH_REQUEST": True,
"MAX_CONTENT_LENGTH": None,
"SEND_FILE_MAX_AGE_DEFAULT": None,
"TRAP_BAD_REQUEST_ERRORS": None,
"TRAP_HTTP_EXCEPTIONS": False,
"EXPLAIN_TEMPLATE_LOADING": False,
"PREFERRED_URL_SCHEME": "http",
"TEMPLATES_AUTO_RELOAD": None,
"MAX_COOKIE_SIZE": 4093,
```

✓ WSGI：为Python语言定义的Web服务器和Web应用程序或框架之间的一种简单而通用的接口

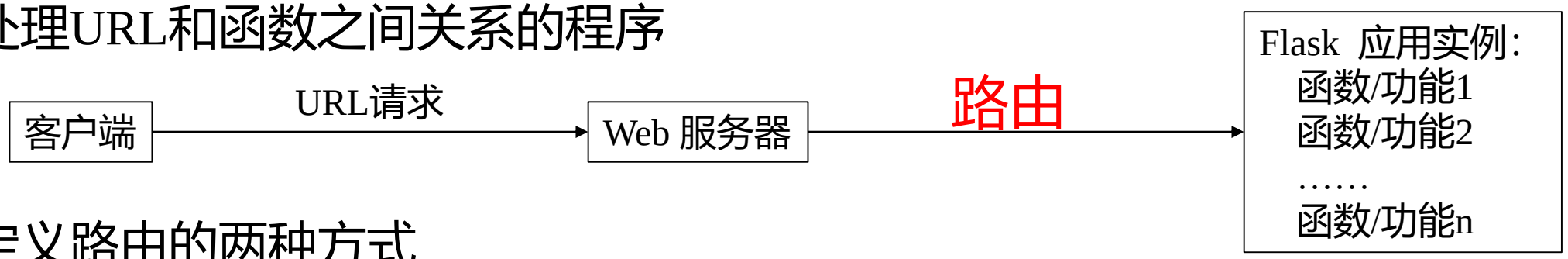
https://blog.csdn.net/u010339879/article/details/100544397

# 应用的基本结构



## 路由

### 处理URL和函数之间关系的程序




### 定义路由的两种方式

- ✓ 装饰器：@app.route(‘/’)；@app.route(‘/hello’)；@app.route(‘/hello/<parm>’)，parm可以指定类型 <类型：形参>，允许的类型有string、 int、 float、path、uuid
- ✓ add_url_rule()函数：app. add_url_rule(url, endpoint, view_function, methods)

  url：函数对应的URL规则，与@app.route一致，必须以“/”开始，建议也以“/”结束

  endpoint：为URL定义的名称，使用url_for()函数可以反转到URL，默认为函数名

  view_function：视图函数名，只需给名称，不要加（）

  methods：指定请求方式，默认是’GET’，设置为[‘POST’,’GET’]表示两种请求方式均可

# 应用的基本结构



- 视图函数
  - 处理请求并给出响应的函数，函数输入为request对象，输出为response对象
  - 实现：

```
def hello():
    return response
```

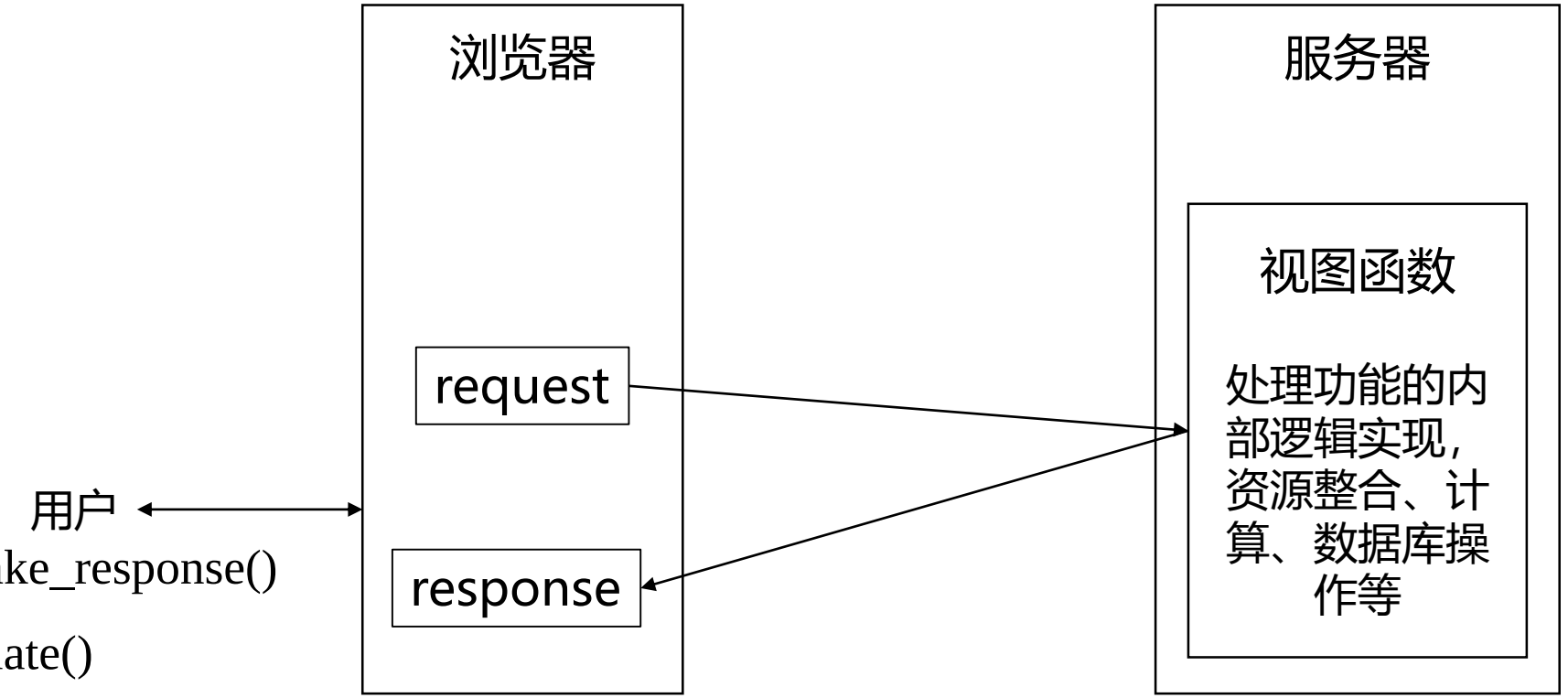
返回值：字符串/html

jsonify

元组

Response/make_response()

render_template()




输入：http://127.0.0.1:5000/requestobject?username=zhansan&password=123456

# 应用的基本结构



- request
  - 请求对象，封装了客户端发出的HTTP请求的全部信息
  - 属性及方法：

| | | | |
|---|---|---|---|
| form | 请求提交的表单字段，字典 | query_string | URL的查询字符串部分，二进制 |
| args | get方式的所有参数字段，字典 | full_path | path+query_string |
| values | form+args，字典 | url | 客户端请求的完整URL |
| cookie | 请求的所有cookie，字典 | base_url | url中除去查询字符串的部分 |
| headers | 请求的HTTP头部，字典 | remote_addr | 客户端的IP地址 |
| files | 请求上传的所有文件，字典 | endpoint | 处理请求的Flask端点名称 |
| methods | HTTP请求方法 | scheme | URL方案(http or https) |
| host | 请求的主机及端口号 | Is_secure() | 判断请求是否是https |
| path | URL的路径部分 | get_data() | 返回请求主体缓冲的数据 |
| blueprint | 处理请求的Flask蓝本名称 | get_json() | 解析请求主体得到的json |

# 应用的基本结构



- response
  - 响应对象，封装了服务器向客户端发出的响应的全部信息
  - 属性及方法：

| | | | |
|---|---|---|---|
| status_code | HTTP数字状态码1xx，2xx，3xx，4xx，5xx | set_cookie() | 为响应添加cookie |
| headers | 类似字典对象，响应的首部 | delete_cookie() | 删除cookie |
| content_length | 响应主体的长度 | set_data() | 使用字符串或字节值设置响应 |
| content_type | 响应主体的媒体类型 | get_data() | 获取响应主体 |

  - 几点说明：
    - ✓ 可以使用make_response()函数构造响应对象；
    - ✓ 响应可以是一个新的URL，使用redirect()来生成；
    - ✓ 响应可以错误，使用 abort()生成

# 应用的基本结构



- 程序运行
  - 在开发环境下，设置程序运行的相关配置。
  - 实现：

```
if __name__ == '__main__':
    app.run(debug=True)
```

```
host: str | None = None,
port: int | None = None,
debug: bool | None = None,
load_dotenv: bool = True,
**options: t.Any,
```

  - ✓ host：ip地址，对应一台服务器，默认只能被本机访问，若需要外网访问需要修改为：0.0.0.0
  - ✓ port：端口号，对应一个程序，多个程序用一个端口，会报端口被占用错误
  - ✓ debug：布尔，是否开启调试模式，默认为false，设置为true，默认开启重载器和调试器

# 应用的基本结构



## MTV

- 在Web开发中，架构模式是一种组织和管理代码的方式，它能够使我们的应用程序更易于理解、扩展和维护。MTV架构是一种常见的Web开发架构，它将应用程序分为三个主要组件：模型(Model)、模板(Template)和视图(View)。
  - ✓ 模型是MTV架构的核心组件，它负责定义应用程序的数据结构和业务逻辑
  - ✓ 模板是MTV架构的表示层组件，它负责生成应用程序的用户界面
  - ✓ 视图是MTV架构的控制层组件，它负责处理用户请求，并根据请求的类型和参数来决定如何处理请求

# 第五章 Flask-安装及基本结构 总结



- 安装　　环境配置流程
- 应用的基本结构　　Hello World、初始化、路由、视图函数、request、response、运行、MTV